

TUGAS 7 – MAIN MEMORY



Disusun oleh:

Nama : Putra Ganda Prayoga

NIM : 2410651057

Kelas : C

Fakultas Teknik

Program Studi Informatika

Universitas Muhammadiyah Jember

2025

1 BAB I – PENDAHULUAN

1.1 Latar Belakang

Memori utama merupakan komponen penting dalam sistem operasi yang berfungsi sebagai tempat penyimpanan sementara bagi program dan data yang sedang diproses oleh CPU. Pengelolaan memori yang tidak efisien dapat menyebabkan pemborosan sumber daya dan penurunan kinerja sistem. Oleh karena itu, sistem operasi menyediakan berbagai mekanisme manajemen memori seperti address space, alokasi kontigu, paging, hingga swapping.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep manajemen memori utama
2. Menganalisis address space dan segmentasi memori
3. Menerapkan algoritma alokasi memori
4. Memahami paging, two-level paging, dan swapping

2 BAB II – DASAR TEORI

2.1 Memori Utama

Memori utama (RAM) adalah memori yang dapat diakses langsung oleh CPU untuk menyimpan instruksi dan data yang sedang dieksekusi.

2.2 Address Space

Address space adalah kumpulan alamat logis yang dapat diakses oleh suatu proses. Setiap proses memiliki address space sendiri sehingga antar proses saling terisolasi.

2.3 Alokasi Kontigu

Alokasi kontigu menempatkan proses pada satu blok memori berurutan. Metode ini sederhana namun rentan terhadap fragmentasi eksternal.

2.4 Paging

Paging membagi memori menjadi page dan frame dengan ukuran tetap sehingga menghilangkan fragmentasi eksternal.

2.5 Two-Level Page Table

Two-level page table digunakan untuk mengurangi penggunaan memori page table pada address space yang besar.

2.6 Swapping

Swapping adalah teknik pemindahan proses secara utuh dari memori utama ke disk untuk memberi ruang bagi proses lain.

3 BAB III – HASIL DAN PEMBAHASAN

3.1 Tugas 1 – Analisis Memory Layout

Pada tugas ini dilakukan pengamatan terhadap pembagian segment memori pada sistem operasi Linux. Program berhasil menampilkan text segment, data segment, BSS, heap, dan stack.

Berdasarkan hasil pengamatan, text segment berada pada alamat memori terendah, sedangkan stack berada pada alamat tertinggi. Heap berada di antara data dan stack serta tumbuh ke arah alamat yang lebih besar. Selain itu, alamat memori berubah setiap kali program dijalankan, yang menunjukkan bahwa sistem menggunakan mekanisme Address Space Layout Randomization (ASLR).

a. Percobaan pada ubuntu

Percobaan dilakukan dengan menjalankan program C pada sistem operasi Ubuntu untuk menampilkan pembagian segment memori proses.

```

putra@DESKTOP-87VNBLO:~/praktikum-memory/tugas1$ ./tugas1
=== ENHANCED MEMORY LAYOUT ===

Text Segment:
main()   : 0x5a4dfb2ab2ca
func1()  : 0x5a4dfb2ab249
func2()  : 0x5a4dfb2ab254

Data Segment:
global_init      : 0x5a4dfb2ae010
static_array[0]  : 0x5a4dfb2ae040
string literal   : 0x5a4dfb2ac030

BSS Segment:
global_uninit    : 0x5a4dfb2ae030

Heap Segment:
heap1            : 0x5a4dfba3b2a0
heap2            : 0x5a4dfba3b2d0

Stack Segment:
main local var   : 0x7ffd40b7c924
Stack level 0 - local_var address: 0x7ffd40b7c904
Stack level 1 - local_var address: 0x7ffd40b7c8d4
Stack level 2 - local_var address: 0x7ffd40b7c8a4

PID: 2359
Tekan Enter untuk lihat memory map...

```

Hasil

Program berhasil menampilkan segment:

- Text
- Data
- BSS
- Heap
- Stack

Hasil `/proc/[pid]/maps` menunjukkan bahwa setiap segment memiliki permission yang sesuai.

Analisis

Text segment berada pada alamat terendah, sedangkan stack berada pada alamat tertinggi. Heap tumbuh ke arah alamat lebih besar dan stack ke arah lebih kecil. Alamat memori berubah setiap eksekusi karena mekanisme ASLR (Address Space Layout Randomization).

b. Memory Map Proses

```
5a4dfb2ac000-5a4dfb2ad000 r--p 00002000 08:30 2532
    /home/putra/praktikum-memory/tugas1/tugas1
5a4dfb2ad000-5a4dfb2ae000 r--p 00002000 08:30 2532
    /home/putra/praktikum-memory/tugas1/tugas1
5a4dfb2ae000-5a4dfb2af000 rw-p 00003000 08:30 2532
    /home/putra/praktikum-memory/tugas1/tugas1
5a4dfba3b000-5a4dfba5c000 rw-p 00000000 00:00 0
    [heap]
76a1d0800000-76a1d0828000 r--p 00000000 08:30 50724
    /usr/lib/x86_64-linux-gnu/libc.so.6
76a1d0828000-76a1d09bd000 r-xp 00028000 08:30 50724
    /usr/lib/x86_64-linux-gnu/libc.so.6
76a1d09bd000-76a1d0a15000 r--p 001bd000 08:30 50724
    /usr/lib/x86_64-linux-gnu/libc.so.6
76a1d0a15000-76a1d0a16000 ---p 00215000 08:30 50724
    /usr/lib/x86_64-linux-gnu/libc.so.6
76a1d0a16000-76a1d0a1a000 r--p 00215000 08:30 50724
    /usr/lib/x86_64-linux-gnu/libc.so.6
76a1d0a1a000-76a1d0a1c000 rw-p 00219000 08:30 50724
    /usr/lib/x86_64-linux-gnu/libc.so.6
76a1d0a1c000-76a1d0a29000 rw-p 00000000 00:00 0
76a1d0a52000-76a1d0a55000 rw-p 00000000 00:00 0
76a1d0a5b000-76a1d0a5d000 rw-p 00000000 00:00 0
76a1d0a5d000-76a1d0a5f000 r--p 00000000 08:30 50721
    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
76a1d0a5f000-76a1d0a89000 r-xp 00002000 08:30 50721
    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
76a1d0a89000-76a1d0a94000 r--p 0002c000 08:30 50721
    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
76a1d0a95000-76a1d0a97000 r--p 00037000 08:30 50721
    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
76a1d0a97000-76a1d0a99000 rw-p 00039000 08:30 50721
    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fffd40b5d000-7fffd40b7e000 rw-p 00000000 00:00 0
    [stack]
7fffd40b9c000-7fffd40ba0000 r--p 00000000 00:00 0
    [vvar]
7fffd40ba0000-7fffd40ba2000 r-xp 00000000 00:00 0
    [vdso]
```

b. Hasil

Sebagian proses gagal dialokasikan meskipun total memori masih tersedia.

c. Analisis

Hal ini terjadi akibat fragmentasi eksternal, di mana sisa memori tersebar dan tidak tersedia dalam satu blok kontigu yang cukup besar.

3.2 Tugas 2 – Contiguous Allocation (Next-Fit)

Pada simulasi alokasi memori menggunakan algoritma Next-Fit, tidak semua proses dapat dialokasikan meskipun total memori masih tersedia. Hal ini terjadi karena sisa memori tersebar dalam beberapa blok kecil sehingga tidak tersedia satu blok kontigu yang cukup besar.

Kondisi tersebut menunjukkan terjadinya fragmentasi eksternal, yang merupakan kelemahan utama dari metode alokasi kontigu.

a. Percobaan pada ubuntu

```
putra@DESKTOP-87VNBLO:~/praktikum-memory/tugas2$ nano next_fit.c
putra@DESKTOP-87VNBLO:~/praktikum-memory/tugas2$ gcc next_fit.c
-o next_fit
./next_fit
Process No      Process Size      Block No
1                212                2
2                417                5
3                112                5
4                426              Not Allocated
```

b. Hasil

Sebagian proses gagal dialokasikan meskipun total memori masih tersedia.

c. Analisis

Hal ini terjadi akibat fragmentasi eksternal, di mana sisa memori tersebar dan tidak tersedia dalam satu blok kontigu yang cukup besar.

3.3 Tugas 3 – Paging

Pada tugas ini dilakukan simulasi translasi alamat menggunakan paging. Logical address dipecah menjadi page number dan offset, kemudian diterjemahkan menjadi physical address menggunakan page table.

Dari hasil pengujian dapat diketahui bahwa paging mampu menghilangkan fragmentasi eksternal. Namun, masih terdapat kemungkinan terjadinya fragmentasi internal akibat sisa ruang pada page terakhir. Offset tidak berubah selama proses translasi alamat.

a. Percobaan pada ubuntu

```
putra@DESKTOP-87VNBLO:~/praktikum-memory/tugas3$ nano paging.c
putra@DESKTOP-87VNBLO:~/praktikum-memory/tugas3$ gcc paging.c -o
paging
./paging
Page Size: 100
Page Table:
Page 0 -> Frame 2
Page 1 -> Frame 4
Page 2 -> Frame 1
Page 3 -> Frame 3
Page 4 -> Frame 0

Masukkan Logical Address: 350

Logical Address: 350
Page Number: 3
Offset: 50
Frame Number: 3
Physical Address: 350
```

b. Hasil

Logical address akan diterjemahkan menjadi physical address menggunakan page table.

c. Analisis

Paging menghilangkan fragmentasi eksternal karena alokasi tidak harus kontigu. Namun masih terdapat fragmentasi internal akibat sisa ruang pada page terakhir. Offset tidak berubah selama translasi alamat.

3.4 Tugas 4 – Two-Level Page Table

a. Percobaan pada ubuntu

Percobaan dilakukan menggunakan simulasi two-level page table untuk menerjemahkan alamat logis.

```
putra@DESKTOP-87VNBLO:~/praktikum-memory/tugas4$ nano two_level_paging.c
putra@DESKTOP-87VNBLO:~/praktikum-memory/tugas4$ gcc two_level_paging.c -o two_level
./two_level
Page Size: 100
Outer Page Table:
Outer 0 -> Inner Table 0
Outer 1 -> Inner Table 1

Masukkan Logical Address: 350

Logical Address: 350
Outer Index: 1
Inner Index: 1
Offset: 50
Frame Number: 5
Physical Address: 550
```

b. Hasil

Translasi alamat dilakukan melalui outer page table dan inner page table sebelum mendapatkan frame fisik.

c. Analisis

Two-level paging mengurangi penggunaan memori page table dengan hanya membuat inner table yang dibutuhkan. Metode ini lebih efisien untuk address space besar.

3.5 Tugas 5 – Swapping

a. Percobaan pada ubuntu

Simulasi swapping menunjukkan proses pemindahan proses dari memori utama ketika kapasitas tidak mencukupi.


```
putra@DESKTOP-87VNBLO:~/praktikum-memory/tugas5$ nano swapping.c
putra@DESKTOP-87VNBLO:~/praktikum-memory/tugas5$ gcc swapping.c
-o swapping
./swapping
Total Memory: 1000

Process 1 (Size 400): Loaded into memory. Memory used: 400

Process 2 (Size 300): Loaded into memory. Memory used: 700

Process 3 (Size 500): Not enough memory. Swapping required.
Process 3 swapped in. Memory used: 500
```

b. Hasil

Ketika memori tidak mencukupi, proses lama dikeluarkan dari memori dan digantikan oleh proses baru.

c. Analisis

Swapping memungkinkan sistem menjalankan lebih banyak proses, namun menyebabkan penurunan kinerja karena aktivitas I/O disk yang tinggi. Swapping bersifat solusi darurat dan jarang digunakan secara penuh pada sistem modern.

3.6 Tugas 6 – Architecture Analysis

Analisis Arsitektur Prosesor

Berdasarkan hasil pengujian menggunakan perintah `lscpu`, sistem yang digunakan berjalan pada arsitektur **64-bit (x86_64)**. Prosesor yang digunakan mendukung pemrosesan multi-core dan multi-thread, sehingga memungkinkan sistem menjalankan beberapa proses secara bersamaan secara efisien. Arsitektur 64-bit memungkinkan sistem mengelola address space yang jauh lebih besar dibandingkan arsitektur 32-bit.

```
putra@DESKTOP-87VNBLO:~$ lscpu | grep -E "Architecture|CPU\(s\)|
Model name|Thread|Core"
Architecture:                x86_64
CPU(s):                      16
On-line CPU(s) list:         0-15
Model name:                   AMD Ryzen 7 7735HS with Ra
deon Graphics
Thread(s) per core:           2
Core(s) per socket:           8
NUMA node0 CPU(s):           0-15
```

Ukuran Page Memori

Pengujian ukuran page dilakukan menggunakan perintah `getconf PAGE_SIZE`. Hasil menunjukkan bahwa sistem menggunakan ukuran page sebesar **4096 byte (4 KB)**. Ukuran page ini umum digunakan pada sistem operasi modern karena memberikan keseimbangan antara efisiensi penggunaan memori dan overhead manajemen paging.

```
putra@DESKTOP-87VNBLO:~$ getconf PAGE_SIZE
4096
```

Translation Lookaside Buffer (TLB)

Informasi mengenai Translation Lookaside Buffer (TLB) diperoleh menggunakan perintah `cpuid`. Dari hasil yang ditampilkan, prosesor menyediakan TLB sebagai cache khusus untuk menyimpan hasil translasi alamat virtual ke alamat fisik. Keberadaan TLB bertujuan untuk mempercepat proses akses memori dengan mengurangi waktu pencarian page table.

```
putra@DESKTOP-87VNBLO:~$ cpuid | grep -i tlb
cache and TLB information (2):
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/ebx):
x):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/ebx):
x):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
  INVLPGB/TLBSYNC hyperv interc enable = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/ebx):
x):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/ebx):
x):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
  INVLPGB/TLBSYNC hyperv interc enable = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/ebx):
x):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/ebx):
x):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
  INVLPGB/TLBSYNC hyperv interc enable = false
L1 TLB information: 1G pages (0x80000019/eax):
```

Hierarki Cache

Hierarki cache pada sistem diperiksa menggunakan perintah `lscpu | grep -i cache`. Hasil menunjukkan bahwa sistem memiliki beberapa level cache, yaitu **L1**, **L2**, dan **L3**. Cache L1 memiliki ukuran paling kecil namun kecepatan paling tinggi, sedangkan cache L3 memiliki

ukuran paling besar dan digunakan bersama oleh seluruh core. Hierarki cache ini berperan penting dalam meningkatkan performa sistem dengan mempercepat akses data yang sering digunakan.

```
putra@DESKTOP-87VNBLO:~$ lscpu | grep -i cache
L1d cache:                256 KiB (8 instances)
L1i cache:                256 KiB (8 instances)
L2 cache:                 4 MiB (8 instances)
L3 cache:                 16 MiB (1 instance)
```

Informasi Memori Utama (RAM)

Kapasitas dan penggunaan memori utama diperoleh melalui perintah `free -h`. Dari hasil tersebut dapat diketahui total RAM yang tersedia serta jumlah memori yang sedang digunakan dan masih bebas. Informasi ini menunjukkan kemampuan sistem dalam menangani beban proses tanpa harus bergantung pada swap.

```
putra@DESKTOP-87VNBLO:~$ free -h
              total        used        free      shared  buff/c
ache  available
Mem:      7.4Gi        386Mi        6.2Gi         3.0Mi         8
34Mi      6.9Gi
Swap:      2.0Gi          0B         2.0Gi
```

Aktivitas Virtual Memory

Aktivitas memori virtual diamati menggunakan perintah `vmstat`. Output yang dihasilkan menunjukkan bahwa sistem operasi Linux secara aktif mengelola memori virtual untuk menjaga stabilitas sistem. Mekanisme ini memungkinkan sistem tetap berjalan dengan baik meskipun beban memori meningkat.

```

putra@DESKTOP-87VNBLO:~$ vmstat 1 5
procs -----memory----- ---swap-- -----io----- -system-
- -----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   c
s us sy id wa st
 0  0       0 6491184 53044 801992    0    0   161   56   47
21  0  1 99  0  0
 0  0       0 6490960 53044 802032    0    0    0    0  169
82  0  0 100  0  0
 0  0       0 6490960 53044 802032    0    0    0    0   66
32  0  0 100  0  0
 0  0       0 6490960 53044 802032    0    0    0    8   95
44  0  0 100  0  0
 0  0       0 6490960 53052 802032    0    0    0   60  142
69  0  0 100  0  0

```

Memory Map Proses

Pembagian address space suatu proses dapat diamati melalui file `/proc/self/maps`. Hasil menunjukkan adanya pemisahan antara **text segment**, **data segment**, **heap**, dan **stack**. Hal ini membuktikan bahwa sistem menggunakan mekanisme virtual memory berbasis paging untuk mengisolasi proses dan meningkatkan keamanan sistem.

```

putra@DESKTOP-87VNBLO:~$ cat /proc/self/maps
5a6532cd4000-5a6532cd6000 r--p 00000000 08:30 1501
    /usr/bin/cat
5a6532cd6000-5a6532cda000 r-xp 00002000 08:30 1501
    /usr/bin/cat
5a6532cda000-5a6532cdc000 r--p 00006000 08:30 1501
    /usr/bin/cat
5a6532cdc000-5a6532cdd000 r--p 00007000 08:30 1501
    /usr/bin/cat
5a6532cdd000-5a6532cde000 rw-p 00008000 08:30 1501
    /usr/bin/cat
5a65398d3000-5a65398f4000 rw-p 00000000 00:00 0
    [heap]
7e5496400000-7e5496428000 r--p 00000000 08:30 50724
    /usr/lib/x86_64-linux-gnu/libc.so.6
7e5496428000-7e54965bd000 r-xp 00028000 08:30 50724
    /usr/lib/x86_64-linux-gnu/libc.so.6
7e54965bd000-7e5496615000 r--p 001bd000 08:30 50724
    /usr/lib/x86_64-linux-gnu/libc.so.6
7e5496615000-7e5496616000 ---p 00215000 08:30 50724
    /usr/lib/x86_64-linux-gnu/libc.so.6
7e5496616000-7e549661a000 r--p 00215000 08:30 50724
    /usr/lib/x86_64-linux-gnu/libc.so.6
7e549661a000-7e549661c000 rw-p 00219000 08:30 50724
    /usr/lib/x86_64-linux-gnu/libc.so.6
7e549661c000-7e5496629000 rw-p 00000000 00:00 0
7e549662f000-7e5496651000 rw-p 00000000 00:00 0
7e5496651000-7e54966a8000 r--p 00000000 08:30 48945
    /usr/lib/locale/C.utf8/LC_CTYPE
7e54966a8000-7e54966a9000 r--p 00000000 08:30 49209
    /usr/lib/locale/C.utf8/LC_NUMERIC
7e54966a9000-7e54966aa000 r--p 00000000 08:30 49212
    /usr/lib/locale/C.utf8/LC_TIME
7e54966aa000-7e54966ab000 r--p 00000000 08:30 48944
    /usr/lib/locale/C.utf8/LC_COLLATE
7e54966ab000-7e54966ac000 r--p 00000000 08:30 49081
    /usr/lib/locale/C.utf8/LC_MONETARY
7e54966ac000-7e54966b3000 r--s 00000000 08:30 50990
    /usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache

```

Penggunaan Swap

Informasi penggunaan swap diperoleh menggunakan perintah `swapon --show`. Swap berfungsi sebagai memori cadangan ketika memori utama tidak mencukupi. Meskipun swap dapat membantu mencegah sistem kehabisan memori, penggunaannya dapat menurunkan performa karena kecepatannya lebih lambat dibandingkan RAM.

```
putra@DESKTOP-87VNBLO:~$ swapon --show
NAME      TYPE      SIZE USED PRIO
/dev/sdc  partition 2G    0B   -2
putra@DESKTOP-87VNBLO:~$ |
```

BAB IV – KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa manajemen memori merupakan bagian penting dalam sistem operasi. Setiap metode memiliki kelebihan dan kekurangan masing-masing. Alokasi kontigu sederhana namun rentan fragmentasi, sedangkan paging dan two-level paging lebih fleksibel dan efisien. Swapping dapat membantu saat memori terbatas, tetapi memiliki dampak negatif terhadap kinerja sistem.